

UNIVERSITY OF MILANO-BICOCCA

Department of Computer Science, Systems and Communication

Master's Degree Programme in Computer Science



**Innovative Management of Service Requests:
System Design, Automation, and
Reccomendation through RAG and
Fine-Tuning of LLMs**

Advisor: Prof. Leonardo Mariani

Master's Thesis by:

Nicolas Guarini

Student ID: 918670

Academic Year 2024–2025

“What I cannot create, I do not understand”
- Richard Feynman

Contents

Introduction	1
1 Overview of the existing System	3
1.1 Service Requests Workflow	3
1.1.1 SR Initiation and Initial State Management	3
1.1.2 Approval and Execution Workflows	6
1.2 Current System Architecture	7
1.2.1 Main Functionalities and Components	7
1.2.2 Interactions and Communication between Customers and Company networks	10
1.3 Problems and core issues of the current system	11
1.3.1 Architectural and Technological Challenges	11
1.3.2 Code and Data Management Issues	13
1.3.3 Database Schema and Data Integrity Issues	14
1.3.4 Maintenance and Operational Challenges	15
2 Architectural Redesign	17
2.1 New System Requirements	17
2.1.1 Catalog Structure and Management	17
2.1.2 Esecuzione delle Service Requests	21
2.1.3 Import delle anagrafiche	22
2.2 Archietttura del nuovo sistema	23
2.2.1 Componenti principali	23
3 Implementation of a Modular and Scalable Solution	27
4 Migrations	28
5 Reccomendation system through RAG and Fine-tuned LLMs	29
Conclusions	30
Bibliography	31

Introduction

The technological evolution of recent decades has profoundly transformed the business landscape, making IT solutions a crucial factor for the success of enterprises. In this context, Elmec Informatica S.p.A. stands out as a significant player in the Information Technology sector in Italy. Founded in 1971 and headquartered in Varese, Elmec combines extensive industry experience with a strong focus on innovation, offering services and solutions that cover a wide range of business needs, offering advanced IT solutions for medium and large enterprises.

The company manages four datacenters (including one Tier IV certified), provides Hybrid IT services, digital workspace and Device-as-a-Service solutions, and is a key partner for cybersecurity and regulatory compliance. With over 450 certified technicians, Elmec integrates cloud technologies (in collaboration with AWS and GAIA-X), promotes environmental sustainability, and stands out for its innovation through its Technological Campus and ongoing investments in professional training.

The primary context of this internship revolves around the redesign and enhancement of the existing *Service Request Portal (SRP)*, which is essential for managing client service requests efficiently.

A Service Requests is a formal request submitted by a customer to obtain specific services or actions within their IT infrastructure. These requests can encompass a wide range of activities, from creating new user accounts in the Active Directory system to managing user permissions, resetting passwords, and deploying software updates.

Various challenges have been identified within the current architecture, particularly concerning the dynamic forms used by customers for submitting the service requests, the management of customizable fields, and the integration with external data sources.

The main objectives of this internship are:

- **Analysis of the Existing System:** Conduct a comprehensive analysis of the current state of the SRP system, identifying weaknesses and inefficiencies that hinder its performance and user experience;
- **Architectural redesign:** Propose and implement a complete architectural redesign of the system;
- **Modular and Scalable Solution:** Design a modular and scalable solution that can accommodate various client-specific configurations;

- **Migrations:** Plan and execute a full migration of databases and other company services/platforms that use SRP.
- **Fine-tuned LLMs integration:** Explore opportunities for integrating specifically fine-tuned LLM systems into the SR workflow.

Chapter 1

Overview of the existing System

The internship project was not focused on creating an entirely new system from scratch, but rather on a deep redesign and restructuring of an existing system developed many years ago using legacy technologies and affected by several issues and critical limitations.

Before discussing the architectural, technological, and implementation choices that were made, it is necessary to carry out a thorough analysis of the current system, in order to understand its behavior, operational flows, technologies used, the main issues to be addressed, how to resolve them, how to prevent them from reoccurring in the future, and how to ensure that the new system is scalable and maintainable.

1.1 Service Requests Workflow

The lifecycle of a Service Request within the current system is orchestrated through a series of defined states and automated processes, involving interactions between various components and human operators. The flow is designed to manage SRs from initiation to completion, handling approvals, execution, and potential errors [14].

In Figure 1.1 [14], a simple and high-level flowchart is shown that illustrates the main flows and scenarios. Instead, in Figure 1.2 a complete and detailed flowchart is shown that illustrates all the details about every flow, including information on the jobs and components that manage the various sections of an SR's workflow [14].

1.1.1 SR Initiation and Initial State Management

A Service Request can be initiated through two primary channels [14]:

- **Via MyElmec Portal:** Clients can submit an SR by completing a dedicated form from the MyElmec portal. Upon saving the form, a ticket is created in the HelpdeskAdvanced (HDA) system with a **QUALIFIED** status, and a corresponding Service Request is generated in SRP with a **NEW** status. These two entities are then linked via the HDA Ticket ID;
- **Via HDA Ticket by Elmec Operator:** Operators have the ability to associate a catalog SR directly with a ticket they are managing within the

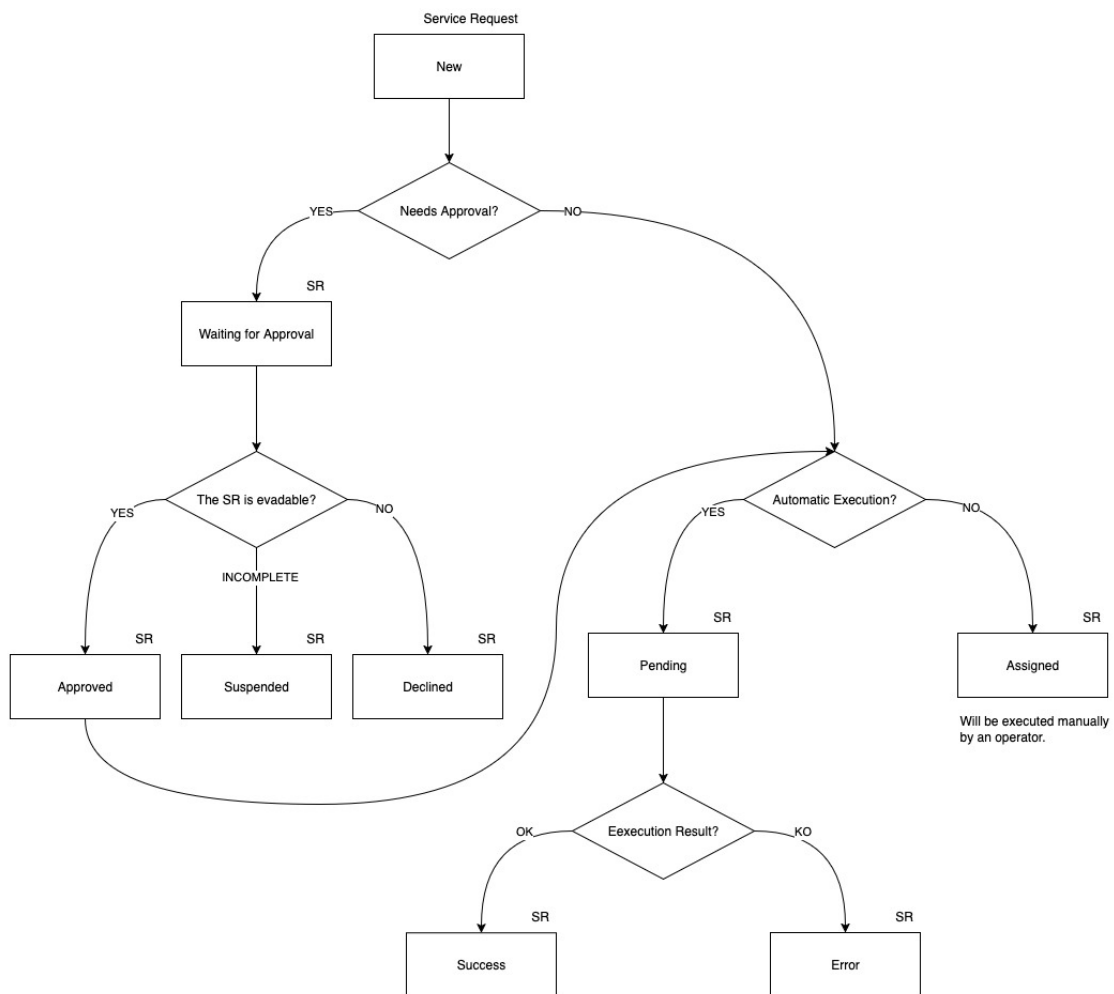


Figure 1.1: High-Level Service Request Lifecycle

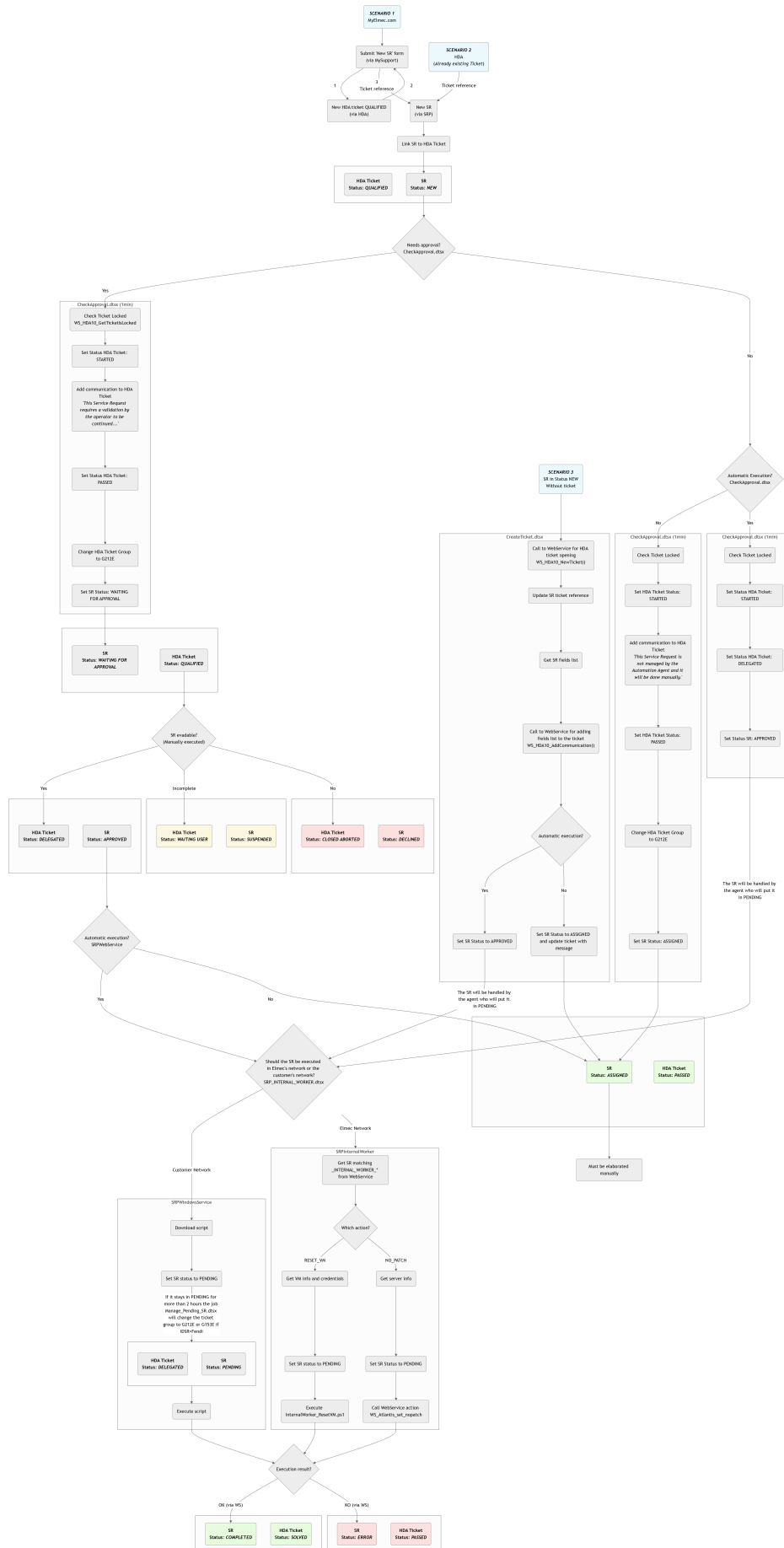


Figure 1.2: AS-IS Complete Service Request Execution Workflow

HDA interface. When the ticket is saved in a `DELEGATED` status, the SR is created in SRP with a `NEW` status, linked to the ticket, and immediately triggers the SR workflow within SRP.

Following its creation, the system evaluates whether the Service Request requires approval by the operators. The logic for setting the initial SR and HDA ticket states is as follows [14]:

- **If approval is required:** the Service Request is set to `WAITING FOR APPROVAL` status, and the corresponding HDA ticket is set to `QUALIFIED`. A communication is added to the ticket, indicating that the SR needs validation from an operator before execution;
- **If no approval is required and it's linked to an automatism:** the Service Request enters a `PENDING` state, and the HDA ticket is set to `DELEGATED`;
- **If no approval is required and it's not linked to an automatism:** the Service Request is `ASSIGNED`, and the HDA ticket is set to `PASSED`. A communication is added to the ticket, noting that the SR requires manual processing by an operator.

1.1.2 Approval and Execution Workflows

For SRs requiring approval (`WAITING FOR APPROVAL` state with an HDA `QUALIFIED` ticket), an operator's intervention is necessary to validate the request. The operator has three possible actions [14]:

- **Cancel the SR:** the HDA ticket is set to `CLOSED ABORTED`, the SR status becomes `DECLINED`, and a notification email is sent to the requesting client;
- **Request client modification:** the HDA ticket is set to `WAITING USER`, the SR status becomes `SUSPENDED`, and a notification email is sent to the client. The workflow pauses until the client modifies the SR via Elmec.com;
- **Approve the SR:** the HDA ticket is set to `DELEGATED`, and the SR status becomes `APPROVED`.

Once a SR is `APPROVED`, SRP manages its subsequent flow based on whether it is linked to an automatism [14]:

- **If the SR is linked to an automatism:** the SR transitions to a `PENDING` state, while the HDA ticket remains `DELEGATED`. This implies it's awaiting automated execution;
- **If the SR is not linked to an automatism:** the SR is immediately set to `ASSIGNED`, and the HDA ticket is moved to `PASSED`. A note is added to the ticket indicating that the SR requires manual processing.

An SR in **PENDING** state is awaiting the execution of its associated automatism. The outcome of this automated execution determines the next state [21]:

- **Successful execution:** the SR is set to **COMPLETED**, and the corresponding HDA ticket is set to **SOLVED**;
- **Unsuccessful execution:** the SR transitions to an **ERROR** state, and the HDA ticket is set to **PASSED**. Communication regarding the error is added to the ticket for visibility within HDA.

The execution of Service Requests themselves can involve the SRP Agent on the client's Domain Controller for operations requiring client-side interaction (e.g., Active Directory or Exchange environment changes). The SRP Agent periodically checks **ELMEC-SRP01** for pending SRs. If found, it retrieves the SR details, loads the relevant PowerShell script, and executes it. The agent then informs **ELMEC-SRP01** that the SR is running [21]. Monitoring is performed by a scheduled **READLOG** task on the client side, which checks the PowerShell script's log file every minute. A "KEY" in the log indicates completion, prompting the SRP Agent to inform **ELMEC-SRP01** to set the SR to **COMPLETED**, triggering an HDA status update. An "ERROR" key indicates failure, leading to the log being sent to **ELMEC-SRP01** and an error signal. A timeout mechanism also signals an error if completion is not detected within a specified duration [21].

1.2 Current System Architecture

The *SRP* system is fundamentally divided into two primary segments:

- Company Network;
- Client Network.

These segments interact and communicate exclusively through a *DMI Console* [16], which acts as a unidirectional bridge, transferring requests from the client network to the internal company network [16] [7].

The overall architecture is depicted in the diagram in figure 1.3 [16].

The system's operational flow involves several key components across both networks, orchestrating the processing of service requests.

1.2.1 Main Functionalities and Components

The current *SRP* system relies on a suite of interconnected components, each serving a specific role in the lifecycle of a service request.

Elmec's Network Components:

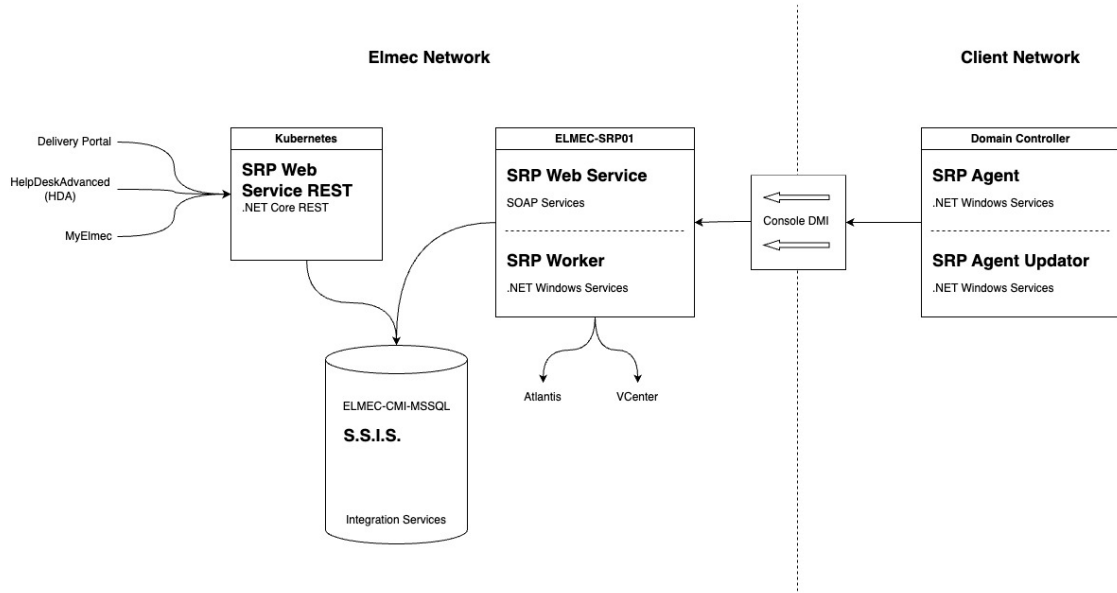


Figure 1.3: AS-IS System Architecture

- **SRP Web Service REST** [20]: This component, deployed on a Kubernetes cluster, exposes .NET Core RESTful services [16]. It serves as the primary interface for external systems such as:
 - **Delivery Portal**: An asset management system used to manage Service Request catalog configuration and Elmec’s internal and client servers;
 - **HDA (HelpDeskAdvanced)**: A ticket management system. Each service request is intrinsically linked to an HDA ticket via an ID, and HDA is responsible for initiating and associating service requests with existing tickets.
 - **MyElmec**: A client-facing portal enabling customers to create, manage, and monitor the various services provided by Elmec (e.g., servers, management software, cloud solutions).

The *SRP Web Service REST* communicates with the *ELMEC-CMI-MSSQL* server.

- **ELMEC-CMI-MSSQL**: This server hosts the central SQL Server Database for the SRP system. It is the persistent storage layer for all service request data and related information [16].
- **S.S.I.S. (SQL Server Integration Services)**: Running on the ELMEC-CMI-MSSQL server, SSIS packages are crucial for data transformation and import/export operations. They facilitate the transfer of data, such as master data (e.g., Active Directory users, Organizational Units, groups) sent by SRP Agents, into the database. Each SSIS package is designed to transform the received information for database insertion or vice versa [16].

- **ELMEC-SRP01:** This Windows Server functions as the core engine for service requests within the Elmec network. It houses all the PowerShell scripts associated with individual service requests, which are maintained by the Platform team [16]. Key components residing on ELMEC-SRP01 include:
 - **SRP WebServiceSoap:** This component provides SOAP services primarily for communication with the SRP Agent located on the client side. It interacts with the database to retrieve necessary information before exposing it to the agents for service request execution [19].
 - **SRP InternalWorker:** A Windows service that operates similarly to the SRP Agent but executes entirely within the Elmec network. This is utilized for service requests that do not require interaction with the client's network or Active Directory (e.g., "no patch" service requests). The InternalWorker queries the database every 10 seconds for "internal worker" type jobs (like removing a server from a patching session, or resetting a managed Virtual Machine), connects to the SOAP service, retrieves the Service Request ID and its fields, and initiates job execution. It also monitors logs every 30 seconds to determine the job's status [18].

The only component in the client's network is called **Domain Controller**, which represents the client's Active Directory domain controller, where the communication tools with Elmec are installed. It hosts:

- **SRP Agent (.NET Windows Service):** Typically, one agent is configured per client domain, specifically for Active Directory (AD) and Exchange-related service requests. This agent performs two main functions [21]:
 - Every 8 hours, it reads master data (AD users, OUs, groups) from the domain controller and sends it to ELMEC-SRP01 for storage via SSIS.
 - Every 5 minutes, it queries ELMEC-SRP01 for pending Service Requests. Upon finding any, it requests detailed information, loads the corresponding PowerShell script, and executes it. After initiating the script, the agent informs ELMEC-SRP01 that the SR is "running" and proceeds to the next SR.
- **SRP Agent Updator (.NET Windows Service):** This service continuously monitors the SRP Agent folder for a file with a .new extension. If detected, it stops the SRP Agent, updates it, and restarts it

1.2.2 Interactions and Communication between Customers and Company networks

The Communication between the company network and the clients' networks is a critical aspect of the SRP system's operations and is handled through the **DMI Console**.

The DMI console serves as a key infrastructural component, primarily acting as a communication bridge between the customer's network and Elmec's internal network. Specifically, for the SRP system, the DMI console is responsible for transferring service requests (SR) from the customer's network to Elmec's internal network, although it does not handle traffic in the opposite direction [7].

Operation of the DMI Console

The connectivity of the DMI appliances to Elmec is established through multiple OpenVPN connections, initiated by the appliance towards the Brunello 4 and Mendrisio Data Centers (for Swiss customers). The DMI consoles are not directly exposed to the Internet and are accessible only by authorized personnel. Hardware requirements for a DMI node include 4 GB of RAM, 4 cores, and 50 GB of disk space [7].

Following recent developments in the DMI, the new consoles use K3S for container management. K3S is a lightweight, certified Kubernetes distribution designed for edge, IoT, and CI/CD environments. It is characterized by being a single binary that reduces external dependencies and uses SQLite as the default database, greatly simplifying installation and management. It already includes essential components such as *containerd* [10].

All consoles are centrally managed via Rancher, and the deployment of the application stack is delegated to ArgoCD, which ensures that container versions are constantly updated. For security reasons, the console in the customer network by default exposes only SSH, while services such as SRP are selectively enabled and only when necessary. All application data present on the consoles is stored on a LUKS (Linux Unified Key Setup) encrypted volume¹, which can only be unlocked if the VPN tunnel is established. The operating system update is performed through a standard Elmec system, called PMD (Patching Management Dashboard) [7].

¹A platform-independent hard disk encryption method that can be used to encrypt disks across a wide variety of tools. This not only ensures full compatibility and interoperability between different software but also guarantees that password management occurs in a secure and documented manner [4].

1.3 Problems and core issues of the current system

The analysis of the architecture and workflow of the current SRP system has revealed a number of critical issues that compromise its scalability, maintainability, and reliability. These problems are deeply rooted in the technological and design choices made years ago and are evident across several key areas of the system, from catalog management to script execution and the import of master data from Active Directory.

1.3.1 Architectural and Technological Challenges

The SRP system is characterized by a fragmented architectural structure and complex interdependencies between its main components, which leads to several critical issues.

Monolithic Frontend Component

The user interface, responsible for navigating the Service Request catalog and completing the associated forms, is an excessively large and complex Vue.js frontend component (nearly 11,000 lines). This architecture makes modifications, extensions, and client-specific customizations exceptionally challenging [12]. The logic for managing dynamic fields, their interdependencies, and the rules for compilation and validation are all handled at the frontend level through a long series of hard-coded conditional statements, such as the following:

```
1 <div v-if="field.desc == 'Domain'">
2   ...
3 </div>
```

This, in addition to mixing business logic with presentation, represents a significant challenge for catalog configuration, as there is no centralized way to manage the fields and their relationships. Every change, therefore, requires a direct intervention on the code, making the system inflexible and difficult to maintain, especially in a context where the operator modifying the catalog is not a programmer and does not know how the frontend code is structured. For instance, if an operator wanted to add a new "Domain" field to a Service Request, this field would need to have the exact description "Domain" and type "domain"; otherwise, the Vue.js component would not render it correctly.

These dynamics make the system fragile and susceptible to frequent errors that waste time and resources for both clients and operators [12].

Overly Generic Backend REST Service

The only information about the fields returned by the backend REST service (`SRPWebServiceREST`), besides the field name and its mandatoriness, is its *type*. It is therefore easy to imagine how this information, which should only be an indication of the component that needs to be rendered (e.g., textbox, select, radio button, etc.), can become an identifier for something much broader.

For example, a field of type `UPN` is a composite field derived from the value of another field, `'Display Name'`, concatenated with an `'@'` symbol, and suffixed with the value from a select field whose options are the client's domains, provided by an external REST service (e.g., `"n.guarini@elmec.it"`, where `"n.guarini"` is the value of the `"Display Name"` field, and `"elmec.it"` is the value of the `"Domain"` select field).

Despite this complexity, the REST service returns only a JSON document of this type:

```
1 {  
2     "idField": 1603,  
3     "descField": "UPN",  
4     "type": "upn",  
5     "mandatory": true  
6 }
```

thus delegating to the frontend the responsibility of interpreting the type and rendering the field correctly.

Dependency on the SRP Agent

The import of data from clients' Active Directories and the actual execution of Service Requests are handled by an agent (`SRPWindowsService`) installed directly on the clients' Domain Controllers. This approach presents several challenges:

- **Security and Permissions:** The agent often requires credentials with elevated privileges (e.g., `domain-admin`), which clients are increasingly reluctant to grant.
- **Reliability and Control:** The configurations on the agent and the Domain Controller are complex and difficult to manage centrally, which can cause execution errors and data synchronization issues.
- **Scalability:** The need to install, configure, and maintain an agent for each client domain represents a significant operational effort. Alternatives such as more modern provisioning systems will be evaluated.
- **Obsolete Communication Patterns:** The unidirectional communication through the *DMI Console* creates a bottleneck and complicates

the management of asynchronous operations. Communication between components in the client's network and the company network is based on polling (e.g., the SRP Agent queries the server every 5 minutes), an approach that introduces latency and inefficiencies.

1.3.2 Code and Data Management Issues

Code and data persistence management is another critical aspect of the system, characterized by a lack of versioning and modern debugging tools.

Unversioned and Undebuggable Stored Procedures

The intensive use of Stored Procedures² in the MSSQL database represents one of the major weaknesses. These procedures contain fundamental business logic and are:

- **Unversioned:** There is no centralized version control system for Stored Procedures. Changes are made directly on the database, making it impossible to track changes, revert to previous versions, or test modifications in separate environments.
- **Difficult to Debug:** Debugging Stored Procedures is a complex and cumbersome process, often requiring direct access and specific SQL server tools, with the risk of affecting the production environment.

Inaccessible and Unversioned SSIS Jobs

SSIS packages³ are essential for data import and export operations (e.g., AD master data). However, these jobs reside physically on the ELMEC-CMI-MSSQL server, and the only way to access and modify them is via a remote desktop. This approach prevents versioning and collaboration, limits accessibility by making maintenance and debugging an onerous and risky activity, and creates a *single-point-of-failure* and a strong coupling between the import/export business logic and the infrastructure.

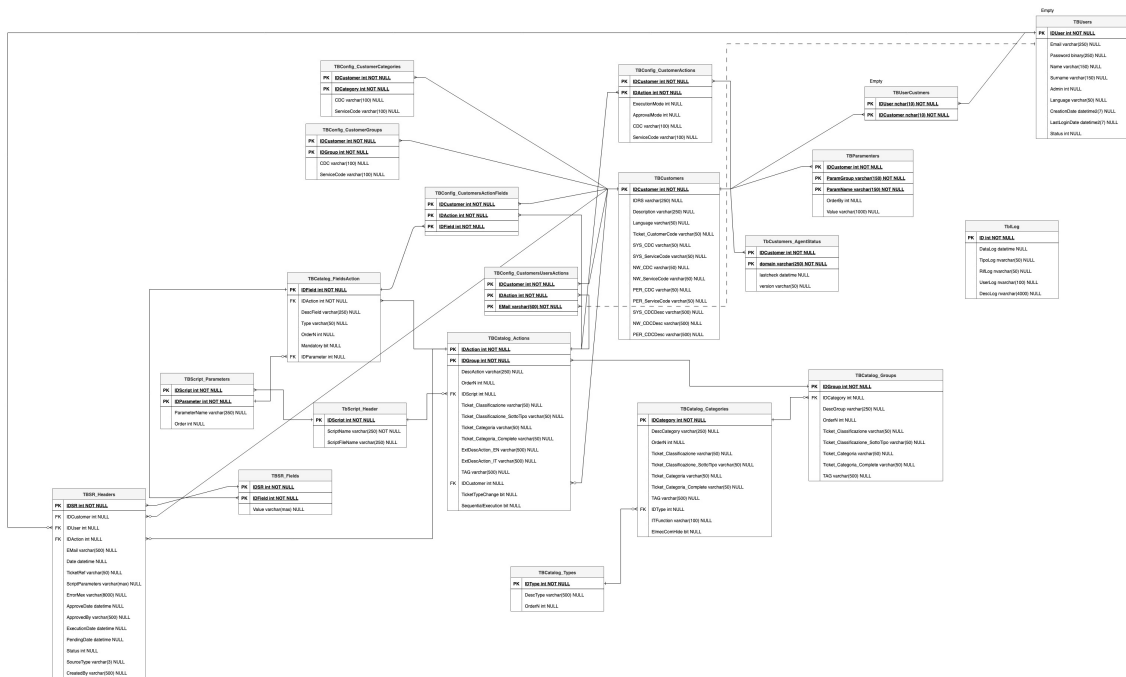


Figure 1.4: AS-IS Database ER Schema

1.3.3 Database Schema and Data Integrity Issues

The analysis of the database schema, represented in figure 1.4, has revealed a number of significant issues that affect the quality and integrity of the data managed by the SRP system.

Lack of Data Integrity and Consistency

One of the most significant problems concerns the lack of explicit referential integrity constraints. In the schema in figure 1.4, although the relationships between tables have been inferred manually, they are not defined at the database level through the use of foreign keys. This absence prevents the database from guaranteeing data integrity [3]: for example, a row in a child table could refer to a non-existent row in the parent table [24]. Such a scenario, known as a "dangling reference"⁴, can generate anomalies and incon-

²Stored procedures are SQL code routines, or sets of instructions, that are pre-compiled and stored within a relational database. They act as logical units of execution, encapsulating business logic directly at the database level [22].

³SQL Server Integration Services (SSIS) is a platform for building data integration and workflow solutions. It is a key component of Microsoft SQL Server, primarily used for Extract, Transform, Load (ETL) operations, enabling the extraction of data from various sources, its transformation, and its loading into different destinations [17].

⁴The "dangling pointer" is a problem typically addressed in the context of low-level programming, where a pointer refers to a memory area that is no longer valid because it

sistencies in the system, making the data unreliable and debugging extremely complex [3].

Data Redundancy

Another issue is the high data redundancy and the large number of duplicate attributes across different tables [21]. This design, which deviates from the principles of normalization [2], leads to storing the same information in multiple locations, which not only increases the data volume but also introduces a high risk of inconsistencies [2].

Unclear and Inconsistent Naming Conventions

The naming convention for attributes and tables is inconsistent and unclear. The schema presents a mixture of conventions (e.g., `IDGroup` vs. `GroupID`, English names mixed with specific terms) and typos (e.g., `TBParamenters`), making it difficult to immediately understand the function of each field. In many cases, the naming is not explicit enough to describe the role of an attribute, requiring an in-depth knowledge of the system to correctly interpret the relationships and business logic. This lack of a unique and standardized vocabulary hinders collaboration among developers, complicates code maintenance, and increases the likelihood of errors.

1.3.4 Maintenance and Operational Challenges

According to information discussed in an internal meeting [15], the daily management of the system is made complex by several inefficiencies and obsolete design choices.

The current system design does not allow for the effective reuse of Service Requests and catalog configurations. Consequently, every modification or addition requires the creation of a new Service Request, even for small variations, leading to a inefficient and difficult-to-manage catalog, consisting of numerous duplicate SRs and fields. This, while functional, is not a scalable approach and leads to significant inefficiencies in catalog management itself. For example, if a Service Request needs to be modified for all clients who use it, the operator will have to modify not only the SR in question but also all the duplicate SRs for each client, with the risk of forgetting a modification or making mistakes. This approach, although probably conceived to prevent error propagation, makes the catalog difficult to navigate and maintain.

has been freed or because the pointer is used outside the context of the variable's existence to which it refers [9], but the same dynamic can also be applied in the context of databases, where the "pointers" are represented by foreign keys.

The new architecture will need to address these challenges by introducing a more robust, versioned system based on modern software design principles and patterns.

Chapter 2

Architectural Redesign

Given the numerous critical issues that emerged from the analysis of the current system, it is clear that a thorough redesign is necessary. This redesign must be able to perform the system's current functions more efficiently and in an optimized manner, while also satisfying a series of requirements that did not exist when the system was developed several years ago, and which the current system cannot manage because it was not designed to do so.

2.1 New System Requirements

The system can be divided into three main components: **Catalog Structure and Management**; **Service Request Execution**, and **Master Data Import**. The requirements for each of these areas will therefore be analyzed.

2.1.1 Catalog Structure and Management

This is the most substantial component of the three, dealing with everything concerning the definition, maintenance, and configuration of the actions executable by each customer. This therefore also includes the management of form fields, customer-specific customizations, scripts, their parameters, the target machine address retrieval logic, and so on.

Before proceeding with the analysis of the requirements, a clarification on the terminology to be used is necessary: an '*Action*' is understood as the formal definition of an operation that a client can execute on its infrastructure; a '*Service Request*' (SR), on the other hand, is understood as the effective execution of an Action. If a parallel with OOP is to be drawn, the Action can be seen as a class, and the Service Request as an object, an instance of the aforementioned class.

Actions

The Action entity can be seen as the core of the catalog component, and it is composed of the following main elements:

- General metadata, such as the name, an extended description, tags, a type, a group, a category, and other information related to administrative

and billing aspects;

- The fields of the corresponding form that must be filled by the client for its execution, including the relative logic tied to types, data validation, dependencies between various fields (e.g., fields that, once completed, "enable" other fields), autofill (e.g., fields whose values are autofilled with values from other fields), sensitive values, and so on;
- The script that will be executed in the customer's infrastructure to perform the effective required operation;
- Script parameters, which may be field values, but also additional parameters necessary for execution (e.g., AWS credentials, 365 credentials, etc...);
- The target machine on which the script must be executed.

Some of these dynamics are already implemented in the current system (with all the critical issues of the case, as already discussed in the previous chapter), while others are not. The crucial point, however, is that all these configuration aspects must be customizable for specific customers and for specific actions [15].

Given that the users of this system are very large clients (Fendi, Valentino, Lavazza, to name a few), it is easy to imagine that each of them has highly specific needs, which make direct reuse of Actions across multiple clients impossible. At the same time, however, the creation of infinite duplicated Actions that share almost everything, with only small differences (as happens in the current system [15]), must be avoided.

It is therefore necessary to implement a system that provides for the possibility of defining a sort of hierarchical structure, where a "parent" action is configured, which can be "inherited" by several child actions that will specify only the modifications, all without affecting the default action.

Campi

Insieme alle action, i campi sono componenti centrali su cui si basa gran parte del sistema. Come già accennato, i campi sono entità che possono essere estremamente diversi tra loro, come campi di testo, select, select a scelta multipla, radio buttons, etc... Nel sistema attuale ci sono 86 tipi di campi diversi. Questo numero è così alto perchè, per come è costruito il sistema attuale, l'informazione "type" è l'unico modo in cui il backend comunica al frontend che tipo di campo dovrà renderizzare [20]. Infatti ci sono molti tipi diversi che però condividono la stessa logica, semplicemente magari usano due servizi diversi per recuperare i dati. Le tipologie effettive di campi sono quindi molte di meno, ed è cruciale identificarle e generalizzarle il più possibile.

L'idea generale è che la logica dei fields dovrà essere spostata lato backend, che

dovrà istituire tutte le informazioni necessarie al frontend per renderizzare la form: dai tipi di campi, alle regex di validazione, ai messaggi di errore, a eventuali servizi esterni da contattare per recuperare la lista di options, piuttosto che per validare i dati, e così via [15].

Tutto questo dovrà ovviamente essere compatibile con i requisiti di personalizzazione specificati nella sezione precedente.

Uno scenario di esempio tipico potrebbe essere il seguente: un customer utilizza una action, ma per qualche motivo ha bisogno di due campi aggiuntivi, che serviranno poi allo script che richiederà a sua volta due parametri aggiuntivi. Per alcuni campi inoltre, il cliente in questione usa regole di validazione diverse e template di autofill specifici. Queste personalizzazioni dovranno poter essere fatte senza modificare l'action e i field di default.

Per quanto riguarda tipi di campi "particolari", il sistema dovrà prevedere:

- Campi select le cui options sono specifiche per il customer (e.g. Campo "Domain", che è una select con tutti i domini del cliente);
- Campi select le cui options sono globali per tutto il sistema (e.g. Campo "Country", che prevede una lista di stati uguali per tutti i clienti);
- Campi select le cui options sono specifiche per il customer e per ottenerle bisogna contattare un servizio esterno, con eventuale possibilità di ricerca (e.g. Campo "Server", che è una select con tutti i server del cliente, restituiti dal servizio del CMDB Atlantis);
- Campi di testo autofillati, ovvero i cui valori sono ottenuti attraverso un sistema di templating (Jinja) che usa i valori di altri campi (e.g. Campo "Logon Name", valorizzato concatenando nome e cognome dell'utente separati da, ad esempio, un punto);
- Campi di testo la cui validazione avviene attraverso regex (e.g. Campo "Date", che deve rispettare il formato standard);
- Campi di testo la cui validazione avviene attraverso un servizio esterno (e.g. Campo "Logon Name", che per essere validato richiede di contattare il relativo servizio di Active Directory per validare che un utente con lo stesso logon name non esiste già);
- Campi con valori sensibili, i cui valori andranno quindi salvati cifrati, usando qualche algoritmo di cifratura simmetrica (e.g. Campo "Password");
- Campi "misti", composti da più "sotto-campi" con caratteristiche diverse (e.g. Campo "UPN", che è composto dal logon name, una chiocciola, e il valore di una select le cui options sono la lista di domini del cliente);

E' importante che il sistema sia progettato in modo scalabile, in modo che se in futuro dovesse aggiungersi un'altra tipologia di campo, lo si potrà implementare con il minimo effort possibile.

Consistenza delle configurazioni e delle esecuzioni

Sarà necessario implementare un sistema di monitoraggio della consistenza delle configurazioni del catalogo [15], in modo da evitare che si possano creare delle configurazioni non valide, che poi porterebbero a errori in fase di esecuzione delle SR.

Ad esempio, se per una action si specifica un campo di tipo `select`, le cui options sono ottenute da parametri specifici per il customer, ma per quel customer non sono stati definiti tali parametri, il sistema dovrà segnalare questa incongruenza tramite un qualche sistema di alerting (e.g. email, ticket, dashboard di monitoraggio). Oppure, se una action è configurata per essere eseguita su un certo tipo di macchina ma tra i campi dell'action non c'è quello necessario per selezionarla, il sistema dovrà segnalare questa incongruenza.

In modo analogo dovrà essere implementato un sistema di monitoraggio della consistenza delle esecuzioni delle SR, in modo da poter segnalare eventuali errori che possono essere accaduti durante l'esecuzione, come ad esempio una SR ferma in uno stato "intermedio" (e.g. `NEW`) per troppo tempo, oppure una SR che ha iniziato l'esecuzione (quindi in `PENDING`) ma non è mai arrivata a completamento (quindi in `COMPLETED` o `ERROR`) entro un certo tempo limite (e.g. 2 ore).

Additional script parameters

Un'altra tematica che dovrà essere gestita è quella dei parametri degli script. Nel vecchio sistema c'era un controllo granulare su ogni singolo parametro, che era legato allo script di una action ed eventualmente a un campo della relativa Service Request. In pratica, a ogni script venivano passati come parametri tutti i campi della action con i relativi valori, più alcuni parametri aggiuntivi non legati ad alcun campo (e.g. `Server Exchange`, `AD Sync Server`, credenziali varie). Questa dinamica dei parametri aggiuntivi avveniva attraverso la definizione di parametri con chiave esterna sul campo impostata a `NULL`, che venivano poi intercettati dalla stored procedure relativa alla generazione della stringa dei parametri che, con degli if con valori hard-coded (e.g. `IF @ParamName = 'ServerExc' BEGIN ...`) venivano valorizzati di conseguenza, recuperando le informazioni con delle logiche custom e non standardizzate.

Nel nuovo sistema, sarà necessario standardizzare queste dinamiche, trovando il modo di poter definire in fase di configurazione delle action i parametri aggiuntivi di cui avranno bisogno oltre a quelli dei campi, e generalizzare il più possibile la logica implementativa, in modo che se si dovessero aggiungere dei nuovi "gruppi" di parametri aggiuntivi (e.g. credenziali AWS, credenziali O365, etc...) potrà essere fatto nel modo più efficiente possibile.

Configurazione della macchina target

Nel vecchio sistema le Service Requests venivano eseguite esclusivamente sui Domain Controller dei clienti. Nello specifico, ogni DC eseguiva un agente, che tramite *polling* controllava periodicamente se ci fossero delle Service Request in stato **PENDING** con il campo **domain** combaciante.

Sarà necessario riprogettare anche questa parte dell'architettura, in quanto poco efficiente e molto limitante per design. Quindi, oltre a riprogettare queste dinamiche sarà anche necessario prevedere la possibilità di scegliere durante la configurazione di una Action, di eseguire le relative SR anche su altri tipi di macchine, come worker interni a Elmec, piuttosto che ad altri tipi di macchine nella rete del cliente diverse dal Domain Controller (e.g. Server Exchange). Dovrà inoltre essere possibile scegliere di eseguire le SR anche attraverso il vecchio flusso, in modo da poter continuare a supportare entrambi i sistemi, almeno inizialmente.

2.1.2 Esecuzione delle Service Requests

Nel sistema attuale, l'esecuzione delle SR avviene secondo questa sequenza di eventi principali:

1. Il frontend invia la richiesta di creazione di una nuova SR al servizio REST (**SRPWebServiceREST**), con tutti i dati necessari (campi compilati, customer, action, etc...);
2. Il servizio REST valida i dati, e se sono corretti crea la SR e i relativi valori dei campi nel database, settandola in stato **NEW**;
3. Un job SSIS, ogni minuto, controlla se ci sono SR in stato **NEW**, e se ne trova le elabora spostandole negli stati successivi (e.g. **WAITING FOR APPROVAL**, **PENDING**, **ASSIGNED**, etc...) a seconda della configurazione della action e del customer;
4. Quando la SR è in stato **PENDING**, significa che è pronta per essere eseguita dall'agente sul DC del cliente;
5. Ogni 10 secondi, l'agente sul DC del cliente controlla se ci sono SR in stato **PENDING** per il suo dominio (attraverso delle chiamate al servizio SOAP (**SRPWebService**));
6. Se ne trova, per ognuna, scarica il relativo script, lo esegue, e aggiorna lo stato della SR a **COMPLETED** o **ERROR** a seconda dell'esito dell'esecuzione (attraverso il servizio SOAP);

E' già stato discusso nel capitolo precedente come questo flusso abbia numerose criticità, che lo rendono inefficiente e poco scalabile. Sarà quindi necessario riprogettarlo completamente, tenendo in considerazione la possibilità di eseguire le SR su macchine diverse dai DC, e la possibilità di eseguirle anche su macchine interne a Elmec, e anche macchine virtuali [15].

Elmec ha già un sistema di *provisioning* interno, chiamato *Provision Prime*, che si occupa di eseguire dei playbook Ansible sulle macchine (interne, esterne, e anche virtuali) [13], e che potrebbe essere sfruttato per questo scopo. Sarà quindi necessario analizzare questo sistema, e capire se e come può essere integrato nel nuovo flusso di esecuzione delle SR, in modo da evitare il polling degli agenti sui DC, e avere un sistema in generale più affidabile ed efficiente [15].

Un limite che va rispettato, però, è che gli script delle action sono tutti in PowerShell, e non è possibile modificarli. Questo perchè sono molto numerosi e sono stati sviluppati negli anni, sono stati testati e validati per ogni singolo cliente, e sono mantenuti da reparti diversi di Elmec. Quindi, anche se il flusso di esecuzione delle SR verrà riprogettato, gli script dovranno essere eseguiti così come sono. Questo ovviamente non vale per i nuovi script che verranno sviluppati in futuro, che potranno essere progettati sfruttando le nuove tecnologie [15].

2.1.3 Import delle anagrafiche

Un altro componente importante del sistema è quello che si occupa di importare le anagrafiche dei clienti, necessarie per popolare alcuni campi delle action (e.g. domini UPN, utenze, gruppi, Organizational Units (OU)¹, databases, etc...). La difficoltà principale risiede nel fatto che queste informazioni risiedono fisicamente nelle macchine dei clienti (stiamo parlando di grossi clienti enterprise, con migliaia di utenze e gruppi), e quindi non è così immediato il loro reperimento in tempo reale.

Nel sistema attuale, questa importazione avviene in modo molto simile allo scenario delle SR, ma al contrario. Avviene attraverso un agente che gira sui DC dei clienti, che periodicamente (ogni ora) (scarica ed) esegue una serie di script PowerShell per recuperare le informazioni necessarie, e le deposita in una cartella specifica del sistema (D:\SRPortal\Anag\). Questi files vengono poi consumati da un job SSIS (uno per ogni tipologia di anagrafica, e.g. `IMP_Anag_AD_Groups.dtsx`) che li importa nel database. I servizi REST espongono poi delle API per recuperare queste informazioni, che vengono usate dal frontend per popolare i campi delle action.

Nel nuovo sistema sarà necessario riprogettare anche questo flusso, in modo da eliminare la dipendenza dagli agenti sui DC dei clienti. Anche in questo caso, però, gli script PowerShell non potranno essere modificati [15], e dovranno essere eseguiti così come sono ora. Sarà quindi necessario analizzare come in-

¹In Windows Server, an Organizational Unit (OU) is a container within Active Directory (AD) used to logically group objects like user accounts, computer accounts, and security groups, similar to a folder in a file system [1].

tegrare l'esecuzione di questi script nel nuovo flusso, magari sfruttando anche in questo caso il sistema di *provisioning* interno di Elmec (*Provision Prime*), in modo da ottenere un sistema più tracciato, affidabile ed efficiente [15].

2.2 Architettura del nuovo sistema

Dopo aver raccolto e definito i requisiti per il nuovo sistema SRM attraverso varie riunioni con gli stakeholder e l'analisi del sistema attuale (SRP), è stato possibile progettare una nuova architettura per il sistema che permetta di soddisfare tali requisiti in modo efficiente e scalabile.

La nuova architettura proposta per il sistema SRM, riportata in figura 2.1, è basata su un'architettura a microservizi, che consente una maggiore modularità, scalabilità e facilità di manutenzione rispetto all'architettura monolitica del sistema SRP attuale [5].

2.2.1 Componenti principali

L'architettura del sistema SRM è composta dai seguenti componenti principali.

Frontend

Saranno sviluppati due frontend web separati per gli utenti finali e per gli amministratori di sistema, entrambi basati su Vue.js. Questi frontend comunicheranno con i servizi REST tramite API HTTP/HTTPS.

Grazie al fatto che la logica di business è stata spostata nei servizi REST, i frontend saranno più leggeri e facili da mantenere, permettendo una maggiore flessibilità nell'implementazione di nuove funzionalità e miglioramenti dell'interfaccia utente. Questo permette anche di integrare un'applicazione mobile in futuro, che potrà utilizzare le stesse API.

Servizi REST

REST (Representational State Transfer) è uno stile architetturale per la progettazione di servizi web che sfrutta i protocolli HTTP per la comunicazione tra client e server [8]. L'idea di base è che le risorse di un sistema (utenti, ordini, documenti, ecc.) vengano esposte attraverso un'interfaccia uniforme, tipicamente HTTP, usando [8]:

- URL per identificare le risorse (`/api/catalog/actions/123/`);
- Verbi HTTP per operare sulle risorse (GET, POST, PUT, DELETE, ...);
- Rappresentazioni delle risorse in formati standard (JSON, XML, ...).

I servizi REST saranno sviluppati utilizzando Django REST Framework, che offre un'architettura robusta e scalabile per la gestione delle API [6].

Questi servizi saranno responsabili della logica di business, della gestione delle richieste degli utenti e dell'interazione con i database e altri componenti del sistema.

Sistema di autenticazione e autorizzazione

L'autenticazione e l'autorizzazione saranno gestite tramite la libreria PyElmecAuth sviluppata internamente, che si interfacerà con il sistema di gestione delle identità aziendale per garantire un accesso sicuro e controllato ai servizi REST.

Sarà quindi necessario integrare in SRM un sistema di permessi granulare per garantire che gli utenti possano accedere solo alle risorse e alle funzionalità per cui sono autorizzati, basato sui ruoli definiti dall'Identity Provider aziendale.

Sistema di caching

Sarà necessario implementare un sistema di caching per migliorare le prestazioni del sistema e ridurre il carico sui database.

In particolare, verranno cachate le risposte delle chiamate a servizi esterni, come ad esempio le API di terze parti necessarie per validare alcuni campi, piuttosto che chiamate per recuperare informazioni sui clienti, per ridurre i tempi di risposta e migliorare l'esperienza utente. Il tempo di vita (TTL) delle cache dovrà essere configurabile in base alle esigenze specifiche di ogni tipo di dato.

Message Broker e Task Queue

Per gestire le operazioni asincrone e migliorare la scalabilità del sistema, verrà utilizzato un Message Broker (ad esempio RabbitMQ, Apache Kafka, Redis, ...) in combinazione con una Task Queue (ad esempio Celery).

Questo è necessario in quanto il ciclo di vita delle SR è molto complesso e prevede molte operazioni che possono richiedere tempi di esecuzione lunghi [14], come ad esempio l'invio di email, il mantenimento dei ticket HDA, l'integrazione con sistemi esterni, ecc. Utilizzando un sistema di messaggistica e code di task, queste operazioni potranno essere eseguite in background, senza bloccare il flusso principale dell'applicazione [23].

Sistema di provisioning

. Sarà anche necessario integrare un sistema di provisioning per eseguire gli script relativi alle SR e quelli di import delle anagrafiche direttamente nella

rete dei clienti. Questo sistema dovrà essere sicuro, affidabile e scalabile, in modo da poter gestire un gran numero di richieste di provisioning contemporaneamente.

Elmec ha già un suo sistema di provisioning chiamato *Provision Prime*, che verrà integrato con il nuovo sistema SRM per eseguire gli script in modo efficiente e sicuro.

Sistema di monitoraggio e logging

Per garantire la stabilità e l'affidabilità del sistema, sarà necessario integrare un sistema di monitoraggio e logging. Questo sistema permetterà di tracciare le performance del sistema, identificare eventuali problemi e analizzare i log per risolvere i bug. Saranno utilizzati strumenti come Prometheus, Grafana, New Relic o altri strumenti simili per implementare questo sistema.

Verranno tracciate sia le metriche di sistema (CPU, memoria, utilizzo del database, tempi di risposta delle API, ecc.) sia i log delle applicazioni (errori, avvisi, informazioni di debug, ecc.), in modo da avere sempre una visione completa dello stato del sistema, ed avere la possibilità di lanciare allarmi automatici in caso di problemi (e.g. se il numero di risposte andate in 500 (Internal Server Error) nell'ultima ora è $\geq 1\%$).

Sistema di containerizzazione e orchestrazione dei container

Per facilitare il deployment, la scalabilità e la gestione del sistema, verrà utilizzato un sistema di containerizzazione (Docker) in combinazione con un sistema di orchestrazione dei container (Kubernetes).

Questo permetterà di isolare le diverse componenti del sistema, semplificare il processo di deployment e garantire una maggiore resilienza e scalabilità del sistema, tramite funzionalità come repliche, cronjob, scaling automatico dei pods, e così via [11].

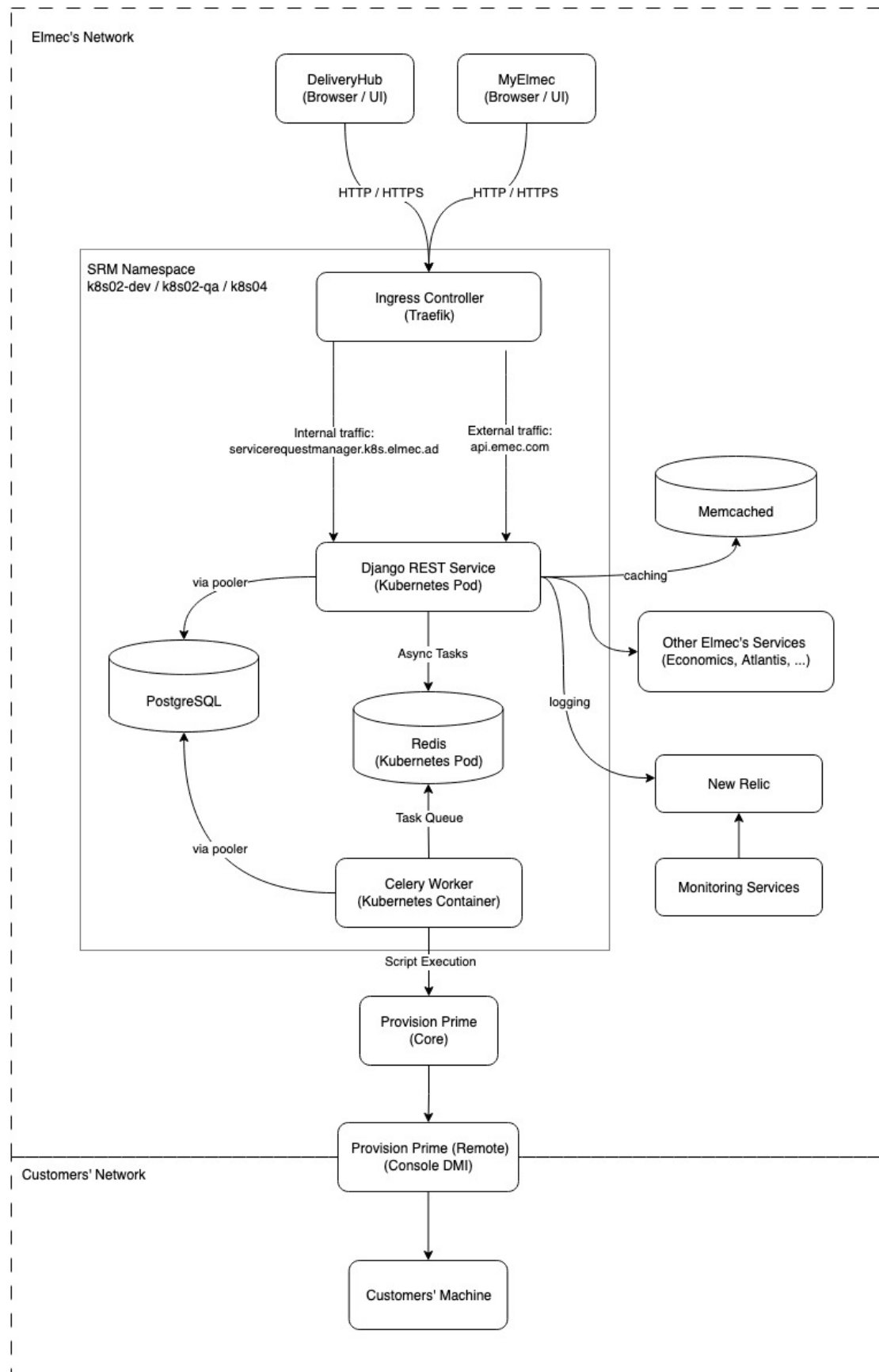


Figure 2.1: SRM System Architecture

Chapter 3

Implementation of a Modular and Scalable Solution

- Overview of technologies to be used (backend, frontend, database)
- Description of tools for automation and containerization
- Reasons behind the choice of said technologies
- Implementation of a modular and scalable solution that can accomodate various client-specific configurations

Chapter 4

Migrations

- Plan and execute a full migration of databases and other company services / platforms that use SRP

Chapter 5

Recommendation system through RAG and Fine-tuned LLMs

- Integration of Retrieval-Augmented Generation (RAG) techniques
- Fine-tuning of Large Language Models (LLMs)
- Implementation of a recommendation system for service requests

Conclusions

- Reflections on the internship experience and acquired skills
- Considerations on the future of the project and potential developments

Bibliography

- [1] *Active Directory Organizational Unit (OU)*. URL: <https://blog.netwrix.com/2024/02/19/active-directory-organizational-unit-ou/> (cit. on p. 22).
- [2] Mashel Albarak and Rami Bahsoon. *Prioritizing Technical Debt in Database Normalization Using Portfolio Theory and Data Quality Metrics*. 2018. arXiv: 1801.06989 [cs.SE]. URL: <https://arxiv.org/abs/1801.06989> (cit. on p. 15).
- [3] Michael R. Blaha. “Referential Integrity Is Important For Databases”. In: 2005. URL: <https://api.semanticscholar.org/CorpusID:6831872> (cit. on pp. 14, 15).
- [4] *Cryptsetup and LUKS - open-source disk encryption*. URL: <https://gitlab.com/cryptsetup/cryptsetup> (cit. on p. 10).
- [5] Lorenzo De Lauretis. “From Monolithic Architecture to Microservices Architecture”. In: *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 2019, pp. 93–96. DOI: 10.1109/ISSREW.2019.00050 (cit. on p. 23).
- [6] *Django REST framework*. URL: <https://www.django-rest-framework.org/> (cit. on p. 24).
- [7] *DMI Infrastructure*. Internal repository, not publicly accessible. URL: <https://git.elmec.com/publicautomation/dmi-infrastructure.git> (cit. on pp. 7, 10).
- [8] Roy T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. University of California, Irvine, 2000 (cit. on p. 23).
- [9] Paul J. Deitel Harvey M. Deitel. *C++. Fondamenti di programmazione*. Apogeo, 2005 (cit. on p. 15).
- [10] *K3s - Lightweight Kubernetes*. URL: <https://docs.k3s.io/> (cit. on p. 10).
- [11] *Kubernetes: Production-Grade Container Orchestration*. URL: <https://kubernetes.io/docs/concepts/overview/> (cit. on p. 25).
- [12] Severi Peltonen, Luca Mezzalana, and Davide Taibi. *Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review*. 2021. arXiv: 2007.00293 [cs.SE]. URL: <https://arxiv.org/abs/2007.00293> (cit. on p. 11).
- [13] *Provision Prime*. Internal repository, not publicly accessible. URL: <https://git.elmec.com/innovation/development/provisionprime.git> (cit. on p. 22).
- [14] Elmec Informatica S.p.A. *Azioni Gestione SRP 4.0*. Internal document, not publicly available., 2016 (cit. on pp. 3, 6, 24).

- [15] Elmec Informatica S.p.A. *Requirements definition meeting for the new system*. Internal meeting. Held with the stakeholders and developers of the old system to gather requirements for the new system. Apr. 2025 (cit. on pp. 15, 18–23).
- [16] Elmec Informatica S.p.A. *Service Request Portal*. Internal document, not publicly available., 2016 (cit. on pp. 7–9).
- [17] *SQL Server Integration Services*. URL: <https://learn.microsoft.com/en-us/sql/integration-services/sql-server-integration-services> (cit. on p. 14).
- [18] *SRP Internal Worker Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srpinternalworker_windowsservice.git (cit. on p. 9).
- [19] *SRP Web Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WebService.git (cit. on p. 9).
- [20] *SRP Web Service REST*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srp_webservicesrest.git (cit. on pp. 8, 18).
- [21] *SRP Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WindowsService.git (cit. on pp. 7, 9, 15).
- [22] *Stored Procedures (Database Engine)*. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine> (cit. on p. 14).
- [23] Jan Tretmans and Louis Verhaard. “A Queue Model Relating Synchronous and Asynchronous Communication”. In: *Protocol Specification, Testing and Verification, XII*. Ed. by R.J. LINN and M.Ü. UYAR. IFIP Transactions C: Communication Systems. Amsterdam: Elsevier, 1992, pp. 131–145. ISBN: 978-0-444-89874-6. DOI: <https://doi.org/10.1016/B978-0-444-89874-6.50015-5>. URL: <https://www.sciencedirect.com/science/article/pii/B9780444898746500155> (cit. on p. 24).
- [24] *What is referential integrity and what does it look like in practice?* URL: <https://www.y42.com/blog/referential-integrity> (cit. on p. 14).